

When Tensor Parallelism Crosses the KV Head Count: Non-Uniform Attention Sharding, Communication Mechanics, and the Batch Frontier for Gemma 4 31B on TPU v6e-8

Rubens Zimbres

ML Engineer / Google Developer Expert in AI and Cloud

rubenszmm@gmail.com

DOI: 10.5281/zenodo.21957969

Abstract. Tensor parallelism divides the weight matrices of a language model across accelerator chips, so each chip streams a fraction of the model per decoding step. The key value cache of attention divides differently: it shards across the attention heads that store keys and values, and a model has far fewer of those than it has weight columns. This study measures what happens on the first parallelism degree that crosses that limit. Gemma 4 31B, a hybrid attention model with fifty sliding window layers and ten global layers, was served with vLLM on a single host TPU v6e-8 slice at tensor parallel degrees four and eight, under a fixed protocol of one variable per experiment and verification of every configuration at the level of the system that acts on it. Three results follow. First, a kernel block size override transplanted from earlier work in this series raised throughput by 69 percent at degree four and 54 percent at degree eight, and a batch ladder on top of the tuned kernel reached 13,540 aggregate tokens per second at batch 1,024 with an eight bit floating point cache verified at the allocation level. Second, per layer trace analysis shows that doubling the parallel degree from four to eight sped up the fifty windowed layers by 1.92 times per layer but the ten global layers by only 1.26 times, collapsing the sliding window cost premium reported earlier in this series from 58 percent to 4 percent: the two layer populations shard through different mechanisms, and the boundary follows the per layer key value head count. Third, device traces show that the communication cost of crossing from a one dimensional to a two dimensional chip mesh changes in structure, not only in size, while a warned all to all penalty never appears as a device operation. Cases in which the serving stack’s advertised configuration differed from its implemented behavior are documented throughout, together with the trace analysis methodology that exposed them.

1 Introduction

A language model that does not fit in the memory of one accelerator chip must be divided across several. Tensor parallelism is the standard division for inference: every weight matrix is split across the chips of a slice, each chip computes its fraction of every layer, and collective operations combine the partial results inside every decoding step. The common summary of this arrangement is that tensor parallelism at degree N makes the per chip work N times smaller. For the weight stream, which dominates the decode step of a large dense model, the summary is close to exact. This paper measures the part of the step for which it is not.

The exception is attention. The key value cache, the stored keys and values that every generated token must read, does not shard along weight columns. It shards along key value heads, and modern models economize on exactly those heads: grouped query attention [1] and multi query attention [2] let many query heads share a few key value heads to shrink the cache. The economy has a consequence for parallel serving that is easy to state and, to our knowledge, had not been measured on this hardware and software stack: the cache can shard at most as many ways as a layer has key value heads. Once the tensor parallel degree exceeds that count, the runtime must do something else with the cache of that

layer, and whatever it does, the layer stops scaling the way the weight stream scales.

Gemma 4 31B makes this crossing observable within a single model. It is a hybrid attention model: fifty of its sixty layers use sliding window attention over a 1,024 token window, and ten use global attention over the full context [3, 4, 5]. The two layer populations also differ in their key value head counts, and the smaller of the two counts is four. A tensor parallel degree of four therefore shards every layer’s cache fully, while a degree of eight crosses the limit for one population and not the other. The serving stack itself announces the crossing: at degree eight its model code prints a warning that layers with four key value heads will be sharded along the head dimension instead, and that this may cause all to all communication overhead. Whether that warning corresponds to a measurable cost, and what the crossing does to the per layer economics of the two attention types, are empirical questions, and this study answers them at the level of compiled kernels and device trace events.

The study is the fifth in a measurement series on TPU inference [6, 7, 8, 9]. The earlier papers established, on a single chip and on a four chip slice, a playbook of deterministic decoding, batch scaling to the arithmetic intensity knee, and a retuning of the decode block size of the ragged paged attention kernel, together with a verification discipline that

repeatedly caught the serving stack accepting configurations it did not apply. This paper carries the playbook and the discipline to an eight chip v6e-8 host and adds the variable the series had not yet isolated: the tensor parallel degree itself.

The contributions are the following.

1. A controlled comparison of tensor parallel degrees four and eight for Gemma 4 31B on one host, one software build, and one session protocol, showing that the decode block size override of the earlier papers transfers to multi chip serving and that its value is large at both degrees: 69 percent at degree four and 54 percent at degree eight.
2. A batch frontier for the tuned configuration on the eight chip slice, from 64 to 1,024 concurrent requests, locating the diminishing returns knee and reaching 13,540 aggregate tokens per second with an eight bit floating point key value cache whose activation was verified at the allocation level rather than at the flag level.
3. A per layer, per attention type decomposition of decode attention cost across the parallelism crossing. Doubling the degree from four to eight speeds up windowed layers by 1.92 times per layer and global layers by 1.26 times. The sliding window premium of 58 percent measured at degree four, which reproduces the finding of the fourth paper in this series on a different slice, collapses to 4 percent at degree eight. Tensor parallelism does not shard this model’s attention uniformly, and the non-uniformity follows the key value head census.
4. A device level account of the communication mechanics of the two mesh shapes: the operation populations, their per instance durations, and the finding that the warned all to all overhead never appears as a device operation in any trace.
5. Documented cases in which the stack’s advertised configuration differed from its implemented behavior, extending the catalogue of the earlier papers, together with a trace analysis method that is robust to a profiler export truncation which would otherwise corrupt every per step statistic.

2 Related Work

Tensor parallelism for transformers was systematized by Megatron-LM [10], which established the column and row partitioning of attention and feed forward weights that inference engines use today. Pope et al. [11] analyzed the efficiency of transformer inference on TPU pods and introduced the cost models for weight streaming and collective communication that later treatments build on; the analytical framing of communication used during the design of this study follows the systems view of Austin et al. [12]. The hardware itself is described in the TPU architecture literature [13].

On the serving engine side, this study runs on vLLM, whose paged key value cache management was introduced

by Kwon et al. [14]. Continuous batching originates with Orca [15], and chunked prefill with Sarathi-Serve [16]; both mechanisms are active in every experiment reported here. DistServe [17] motivates the separation of prefill and decode concerns that this study adopts by reporting decode statistics separately. Kernel level attention optimization on GPUs is exemplified by FlashAttention [18]; the TPU counterpart in this stack is the ragged paged attention Pallas kernel whose block size parameter the present study manipulates.

The attention architecture features at the center of the study have their own lineage. Multi query attention [2] and its generalization to grouped query attention [1] reduce the key value head count to shrink cache size and bandwidth. Sliding window attention as a layer type comes from the long document literature [3] and entered mainstream open models with Mistral 7B [4]; Gemma models interleave windowed and global layers [5]. The interaction measured here, between the key value head count of these architectures and the tensor parallel degree of the serving deployment, is treated in the analytical literature as a sharding constraint [11, 12]; this paper contributes trace level measurements of what a production serving stack does at the crossing and what the crossing costs per layer type.

The four earlier papers of this series supply the instruments and the baselines. The first [6] established the measurement protocol and the block size finding on a single v5e chip. The second [8] carried the tuned kernel to a v6e-4 slice and measured batch scaling and goodput for the same model studied here. The third [7] isolated the decode block size heuristic as the causal variable. The fourth [9] decomposed attention cost per layer type on hybrid models and measured the sliding window premium at realistic context lengths. This paper varies the one factor those papers held fixed.

3 Background: What Shards and What Does Not

A decode step of a dense transformer moves three kinds of bytes through each chip: the weight matrices, the key value cache, and the activations exchanged between chips. Tensor parallelism treats them differently.

Weights. Every weight matrix is partitioned across the N chips of the slice. Each chip stores and streams $1/N$ of the model per step, so the weight stream time falls in proportion to N as long as the interconnect can combine the partial results quickly enough. For Gemma 4 31B in bfloat16 the full weight set is 58.25 GiB; at degree eight each chip streams roughly 7.3 GiB per step, at degree four roughly 14.6 GiB.

Key value cache. Attention partitions across heads. The query heads split freely, but the cache is indexed by key value heads, and a layer with H_{kv} key value heads offers only H_{kv} natural shards. The effective sharding degree of a layer’s

cache is therefore

$$\text{shards}(\ell) = \min(N, H_{kv}(\ell)). \quad (1)$$

Below the limit, per chip cache bytes fall with N like the weights. At and beyond the limit, the runtime must choose an alternative: replicate the cache, pad the heads, or split each head’s feature dimension across chips. The stack under study chooses the last option and says so: at degree eight its Gemma 4 model code logs that layers with $H_{kv} = 4$ will shard the key value tensors along the head dimension, adding that this may introduce all to all communication.

The model under test. Gemma 4 31B has sixty decoder layers in a repeating pattern of five sliding window layers (window 1,024 tokens) followed by one global layer, giving fifty windowed and ten global layers. The configuration as reported by the model library associates two key value head counts with the two populations, sixteen and four, with four as the minimum; the reported census itself depended on the model library version, which is one reason this study treats measured per layer scaling, not configuration printouts, as the census of record. At tensor parallel degree four, Equation 1 gives full sharding for every layer. At degree eight, the sixteen head population still shards fully while the four head population crosses the limit. The model is therefore its own controlled experiment: sixty layers experience the same batch, the same schedule, and the same interconnect, and differ in exactly the property Equation 1 depends on.

The prediction. If the crossing matters, doubling the degree from four to eight should speed up the fully sharded layers by close to two per layer and the head limited layers by materially less, and the gap should be visible in per layer device time with no change to any other variable. Sections 6.4 tests this prediction directly.

Communication. Tensor parallelism pays for its sharding with collectives inside every layer: the partial results of row partitioned matrices must be summed across chips. On a TPU slice the physical arrangement matters. The degree four deployment in this study maps to a one dimensional mesh of four chips; the degree eight deployment maps to a two dimensional mesh of two by four chips, and the compiler is free to implement its collectives hierarchically across the two axes. Section 6.5 reports what operations actually appear on the devices in each case.

4 Experimental Setup

Hardware. All experiments ran on a single ct6e-standard-8t host: one TPU v6e-8 slice of eight chips with 31.24 GiB of usable high bandwidth memory per chip, attached to one virtual machine. The slice is a single host; no data center network is involved, and all collectives travel the on board inter chip interconnect. Durability was provided by a cloud storage

Table 1: Pinned software stack. An environment satisfying the packages’ declared constraints with a newer model library failed at engine initialization; the pins below are the verified working set.

Component	Version
vllm-tpu	0.26.0
tpu-inference	0.26.0
transformers	5.8.1 (pinned; 5.15.0 fails)
jax / jaxlib	0.10.2
libtpu	0.0.43
flax	0.12.4

bucket to which results, traces, and the project code were synchronized, since the virtual machine itself was deleted at the end of each session.

Software stack. Table 1 lists the pinned stack. The pins are part of the findings, not housekeeping. The first environment built for this campaign satisfied every declared version constraint of every package and did not run: the model library at its then current version changed the attribute surface that the TPU inference plugin reads during Gemma 4 model construction, and engine initialization failed before the first token. The working combination required pinning the model library two minor versions below the newest release that the constraint solver had accepted. A dependency specification that is satisfied is not a dependency specification that works, and a study that intends to be reproduced must publish the exact set. The full frozen environment accompanies the archived project code.

Model and workload. The model is Gemma 4 31B in bfloat16 (58.25 GiB of weights), served with a maximum model length of 2,048 tokens. The workload is the fixed protocol of the series: a set of short prompts repeated to the target concurrency, greedy decoding at temperature zero, and exactly 128 generated tokens per request, so that every run executes the same number of decode steps and aggregate throughput is comparable across runs. Prefix caching, continuous batching, and chunked prefill run at engine defaults. The numbers reported are decode dominated; time to first token for the whole batch is recorded and reported but not optimized.

Parallel configurations. Two tensor parallel degrees were measured. Degree four runs on a one dimensional mesh of four chips, leaving four chips of the slice idle; degree eight runs on a two by four mesh using the full slice. Degree two was probed and, as expected from the weight arithmetic, does not fit: 58.25 GiB of weights across two chips exceeds the per chip memory before any cache is allocated. All comparisons between degrees use the same host, build, and session protocol.

The kernel variable. The attention kernel is the stock ragged paged attention Pallas kernel of the TPU inference plugin. Its decode block size is chosen by a built in heuristic that, for this model and context length, compiles 2,048 token key value blocks; the earlier papers of this series showed that for short context workloads this loads mostly masked padding and that overriding the decode block size to 128 tokens recovers large throughput [7]. The override is applied through the vendored kernel harness of the first paper [6]: the kernel source is copied into the project, injected before engine construction, and verified live. Runs labeled heuristic use the untouched stock path.

Run matrix. The campaign comprised, at each degree, a stock heuristic configuration and a block 128 configuration at batch 256, with repeats; a batch ladder at degree eight over 64, 128, 256, and 512 concurrent requests on the tuned kernel; profiled repeats of three configurations for trace analysis; and a final run at batch 1,024 combining the tuned kernel, an eight bit floating point key value cache, and an adjusted memory fraction. A separate anchor run at batch 512 bridged the final session to the campaign sessions; it reproduced the campaign figure within 0.15 percent, which bounds session to session drift for the cross session comparisons below.

5 Methodology and Verification

The series applies three rules, and this campaign added trace analysis machinery that the rules turned out to require.

One variable at a time, same session comparisons. Every claimed effect is a difference between two runs that differ in one configuration item, executed in the same session on the same build. Repeats quantify run to run noise: the per configuration coefficient of variation in this campaign ranged from 0.0 to 1.6 percent, and every reported effect exceeds it by an order of magnitude or more. Output digests are recorded per run; repeats of the same configuration produced statistically equivalent throughput with output variation consistent with the platform’s statistical rather than bitwise determinism, a property established in the fourth paper.

No throughput from profiled runs. Tracing adds overhead that depends on event counts, and event counts depend on the treatment; the first paper documented a case where this inverted a conclusion. Profiled runs in this campaign exist only to feed trace analysis, and their throughput is never quoted.

Verify at the layer that acts. A configuration is evidence only at the level of the system that consumes it. The series orders the levels as a hierarchy: a flag is weaker than the engine’s computed prediction, which is weaker than the compiled artifact observed in a trace or an allocation line

observed in the engine log. This campaign supplied a further illustration. A memory setting exported by the launch tooling did not reach the engine in several runs because of a naming mismatch between scripts, each of which looked correct in isolation. The discrepancy was localized by reading the engine’s own startup echo of the constructor arguments it had received, where the setting was absent. The general form of the rule is that the receiving system’s report of its configuration outranks every layer of tooling above it.

Trace analysis. Device traces were captured for one profiled run per configuration of interest and analyzed with a purpose built parser. Three properties of the trace export shaped the parser, and each is a methodological finding in its own right.

First, the exported trace interleaves host side events (Python threads, queue waits, framework bookkeeping) with device events on separate process tracks; summing them together makes device work appear to be a few percent of the total. The parser attributes only device track events and reports the excluded host time explicitly. Second, compiled program wrapper spans contain the kernels they launch as children; counting wrappers alongside leaves double counts the hierarchy, so wrappers are excluded from attribution. Third, and most consequential: the trace export truncates at a fixed event count, and the truncation is silent. The traces in this campaign captured roughly seven decode steps of 127 at degree eight and roughly fourteen at degree four. Any statistic formed by dividing a raw sum by the requested step count is therefore wrong by the truncation ratio. The parser instead exploits the structure of the program: each per layer operation instance executes exactly once per decode step on each chip, so its mean duration per firing is its true per step cost regardless of where the export stopped. All per layer and per operation figures below are such truncation robust means, computed over dozens of samples per operation (steps times chips), with per operation spreads near one percent. Operations that fire at other cadences, such as prefill bucket kernels, are identified by their execution counts and reported separately.

Framing of throughput. All throughput figures are aggregate slice numbers: tokens per second produced by the whole deployment. Where a per chip figure aids comparison it is stated as the aggregate divided by the chip count. Aggregate multi chip figures are not directly comparable to the single chip figures of the first paper in this series, and no such comparison is made.

6 Results

6.1 The block size override at both parallel degrees

Table 2 and Figure 1 give the campaign’s controlled comparison. At batch 256 the stock heuristic serves 4,752 tokens per second at degree four and 7,052 at degree eight; the block 128

Table 2: Campaign results at batch 256, greedy decoding, 128 tokens per request. Aggregate tokens per second; n is the number of unprofiled repeats; cv is the coefficient of variation across repeats. ITL is mean inter token latency.

TP	Kernel	Tokens/s	cv	ITL (ms)
4	heuristic (bkv 2048)	4,752	–	51.7
4	override (bkv 128)	8,022	0.5%	29.6
8	heuristic (bkv 2048)	7,052	0.0%	33.7
8	override (bkv 128)	10,829	1.6%	21.0

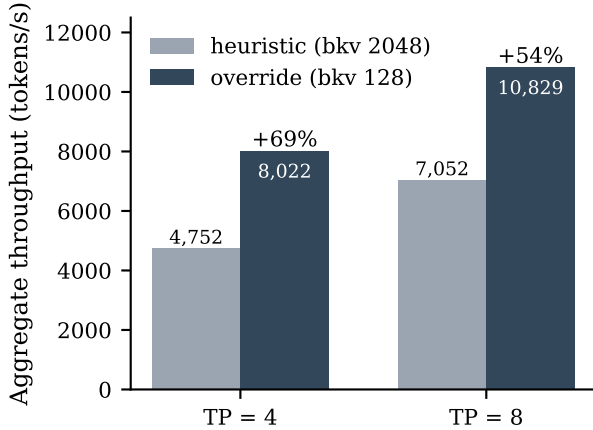


Figure 1: Aggregate throughput at batch 256 for the stock block size heuristic and the 128 token override, at tensor parallel degrees four and eight. The override transfers to multi chip serving with large gains at both degrees.

override raises these to 8,022 and 10,829, gains of 69 and 54 percent. Both gains exceed run to run noise by two orders of magnitude. The compiled kernel names in the traces certify the treatment at the strongest level of the hierarchy: decode kernels in override runs are named for 128 token key value blocks, and in heuristic runs for 2,048 token blocks. The trace also resolves an ambiguity a summary statistic would hide: traces contain a second block size population belonging to prefill kernels, which legitimately use larger blocks; only the decode kernel names carry the treatment.

Two readings follow. First, the block size finding of the earlier papers is not a single chip artifact: it transfers to tensor parallel serving, on a different slice shape, at both measured degrees. Second, the relative gain is smaller at the higher degree (54 versus 69 percent) even though the absolute throughput is larger. Section 6.4 locates the reason in the layer population whose attention cost the override reduces most.

Doubling the parallel degree on the stock heuristic raises throughput by 48 percent (4,752 to 7,052); on the tuned kernel by 35 percent (8,022 to 10,829). Neither is close to the doubling that a uniformly sharded step would allow, and at batch 256 part of the shortfall is simply that the larger deployment is further from its own saturation batch. The batch frontier of the next subsection and the per layer analysis

Table 3: Batch ladder at TP=8 with the block 128 kernel. The batch 1,024 row additionally uses an fp8 key value cache and a memory fraction of 0.85 (Section 6.3); it changes two variables relative to the row above and is reported as a combined effect. TTFT is time to first token for the entire batch.

Batch	Tokens/s	ITL (ms)	Batch TTFT (ms)
64	5,329	9.3	354
128	8,036	13.6	313
256	10,829	21.0	369
512	12,500	37.3	512
1,024	13,540	69.9	808

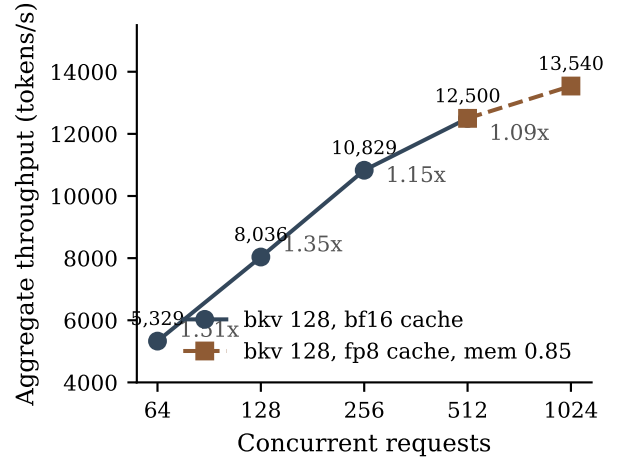


Figure 2: The batch frontier at TP=8 with the tuned kernel. Labels between points give the throughput factor per batch doubling. The final point combines batch 1,024 with an fp8 cache and an adjusted memory fraction.

of Section 6.4 separate those two causes.

6.2 The batch frontier at degree eight

With the kernel variable fixed at the override, the batch was doubled repeatedly on the eight chip deployment. Table 3 and Figure 2 report the ladder. Throughput grows by factors of 1.51, 1.35, and 1.15 per doubling from 64 to 512 requests: the textbook signature of a step whose fixed weight streaming cost is being amortized while its batch linear attention cost grows. The knee is at hand by batch 512.

The batch 1,024 rung initially did not run at all. At the default memory fraction the engine loaded weights and cache and then failed while loading a 1.50 GiB compiled program for structured decoding against 0.71 GiB of remaining per chip memory. The size is explained by the batch itself: a tensor of 1,024 padded requests by the model’s roughly 262,000 entry vocabulary in 32 bit occupies about 1.07 GiB before overhead, and the program must coexist with weights and cache. Lowering the engine’s memory fraction from its 0.92 default to 0.85 released the headroom, and the run completed at 13,540 aggregate tokens per second, 1,693 per chip, at

69.9 ms inter token latency. The 512 to 1,024 step factor of 1.09 continues the diminishing sequence: batch scaling on this deployment is finished, and further aggregate throughput must come from the step itself. Because this run also enabled the fp8 cache, its gain over the 512 row is a combined effect of two variables; the decomposition is left to a planned session (Section 9) and no per variable attribution is claimed.

6.3 The fp8 key value cache, verified at the allocation

Every campaign run’s startup log printed a banner announcing that an eight bit floating point key value cache would be used automatically. Every campaign run’s allocation line, several hundred lines later, recorded a bfloat16 cache: 187,136 cacheable tokens in 1,462 blocks per layer. The banner and the allocation disagreed, and by the verification hierarchy the allocation wins; the campaign therefore ran on a sixteen bit cache throughout, and the earlier papers’ finding that this flag path can be silently inert reproduced on a new stack version.

The final run closed the gap by passing the cache dtype explicitly through the engine constructor argument rather than relying on the automatic path. The allocation line then changed in every dimension a real eight bit cache must change: dtype float8_e4m3fn, capacity 288,000 tokens, 2,250 blocks per layer, per chip memory at 26.57 of 31.24 GiB. The capacity arithmetic closes exactly: halving cache bytes doubles token capacity, and the simultaneous memory fraction reduction from 0.92 to 0.85 scales it by $0.85/0.92$, predicting a factor of $2 \times 0.924 = 1.85$ over a same fraction bf16 cache, or 1.54 over the campaign’s 0.92 fraction bf16 figure, which is the observed $288,000 / 187,136$. One further detail belongs in the record: the banner had announced the e5m2 variant of the format, and the allocation delivered e4m3fn. The configuration that acts differed from the configuration that was advertised twice in the same feature, once in whether and once in which.

6.4 Non-uniform sharding: the crossing, measured per layer

The centerpiece measurement is the per layer decode attention cost by layer type, extracted from device traces as truncation robust means (Section 5). The hybrid architecture makes the kernel names self labeling: the fifty windowed layers compile attention kernels whose names carry the window, and the ten global layers compile the variant without it. Table 4 and Figure 3 give the results for three configurations.

Three findings are in this table.

The premium reproduces, then collapses. At degree four on the tuned kernel, a windowed layer costs 58.1 percent more per decode step than a global layer. This reproduces, on a different slice shape and parallel degree, the sliding window premium that the fourth paper of this series measured and bounded on v6e-4 [9]. At degree eight, on the same kernel, the same model, the same batch, and the same session protocol,

Table 4: Per layer decode attention cost by layer type, in microseconds per layer per decode step (truncation robust means over all captured steps and chips). Premium is the windowed cost relative to the global cost. Totals are per step sums over the fifty windowed and ten global layers.

	TP 4 bkv 128	TP 8 bkv 128	TP 8 heuristic
Windowed ($\mu\text{s}/\text{layer}$)	179.0	93.3	227.9
Global ($\mu\text{s}/\text{layer}$)	113.2	89.5	249.1
Premium	+58.1%	+4.3%	−8.5%
Decode attention (ms/step)	10.08	5.56	13.89

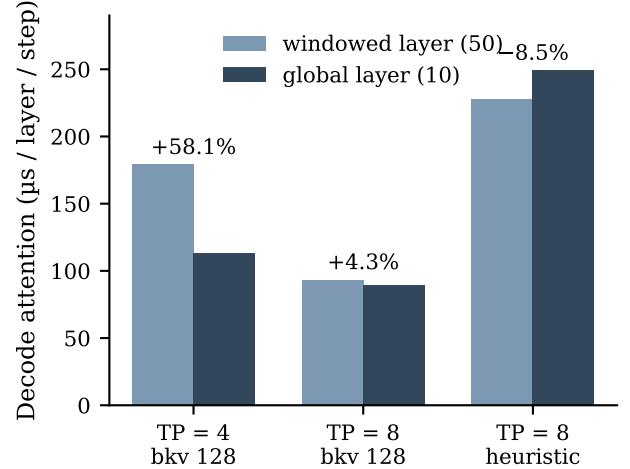


Figure 3: Per layer decode attention cost for windowed and global layers under three configurations. The sliding window premium of 58 percent at TP=4 collapses to 4 percent at TP=8 on the same kernel, and inverts under the stock heuristic’s 2,048 token blocks.

the premium is 4.3 percent. A cost asymmetry that survived a chip generation and a normalization audit in the earlier work does not survive a change of tensor parallel degree.

The collapse is a scaling asymmetry, and it follows the head census. Figure 4 shows why the premium collapsed. Doubling the degree sped up the windowed layers by $179.0/93.3 = 1.92$ times per layer, close to the ideal factor of two, while the global layers gained $113.2/89.5 = 1.26$ times. The boundary between near ideal and degraded scaling coincides exactly with the layer type boundary, which under this model’s configuration is also the key value head boundary of Equation 1: the population that scales near ideally is the one whose sixteen key value heads still shard fully at degree eight, and the population that gains only 1.26 times is the one whose four heads force the head dimension sharding path the engine warned about at startup. The measured split is the prediction of Section 3, observed per layer. Because the configuration printout that assigns head counts to layers itself varied with the model library version, the scaling signature in the trace is the census of record, and it is unambiguous: sixty layers under identical conditions, split into exactly the two predicted scaling classes along the architectural boundary.

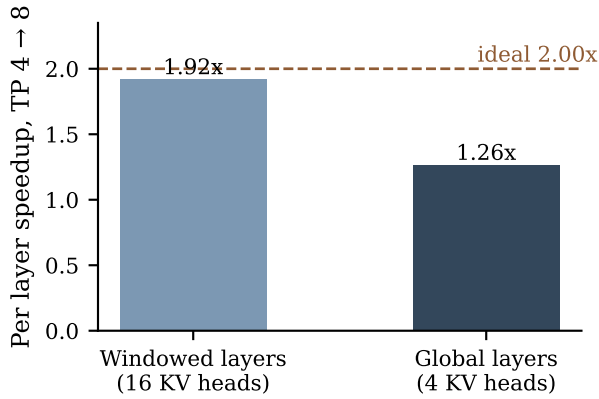


Figure 4: Per layer speedup of decode attention when the tensor parallel degree doubles from four to eight, by layer type, on the block 128 kernel. The population with sixteen key value heads scales near the ideal factor of two; the population with four heads, which crosses the sharding limit of Equation 1 at degree eight, gains 1.26 times.

The premium is a property of a pair, not of a layer type. Under the stock heuristic’s 2,048 token blocks at degree eight, the premium is negative: global layers cost 8.5 percent more than windowed ones. With large blocks, a global layer reads the full context while a windowed layer masks beyond its 1,024 token window, so the windowed layer does less useful reading per block; with 128 token blocks the masking advantage shrinks and the per block overheads dominate. The sign of the windowed versus global cost difference is therefore set jointly by the kernel block size and the window, and any statement of the premium must name both.

The table’s totals also complete the account of Section 6.1. On the heuristic, decode attention costs 13.89 ms per step at degree eight; the override reduces it to 5.56 ms, a factor of 2.5 on the operation the earlier papers identified as the step’s largest batch linear term. And the whole deployment level asymmetry is now attributable: total decode attention fell by 1.81 times when the degree doubled (10.08 to 5.56 ms), against the ideal 2.0, with the entire shortfall contributed by the ten head limited layers. Tensor parallelism sharded this model’s attention, but not uniformly, and the non-uniformity is the measured cost of crossing the key value head count.

6.5 Communication mechanics

The device traces record every collective operation the compiled programs execute, and the two parallel degrees produce qualitatively different populations. The account below is structural; per step communication totals are not quoted, both because the profiled runs on which they would rest inflate collective durations with synchronization waiting, and because the truncated traces capture few steps (Section 8).

At degree four, on the one dimensional mesh, the collective population is uniform: roughly 125 all reduce instances per compiled decode program, one small pair per layer plus a

handful at the step boundary, with per instance means near 40 microseconds, accompanied by only two small all gather instances and no collective permute operations at all. The compiled programs implement the per layer combines as plain single axis all reduces on a four chip ring.

At degree eight, on the two by four mesh, the population changes in structure. The per layer all reduces persist but run at 60 to 74 microseconds per instance, and they are joined by two new populations: roughly thirty all gather instances of 16 to 27 microseconds, and a full complement of collective permute start and done pairs threaded through the program. Collective permutes are the compiler’s point to point primitive; their appearance, together with the higher per instance all reduce durations, is consistent with hierarchical collectives decomposed across the two mesh axes. In the family shares of leaf device operations, collectives account for roughly 13 percent of attributed device time at degree four and roughly 26 percent at degree eight on the same kernel and batch, with attention near 43 and 41 percent respectively; these shares carry profiling overhead and are comparable within a trace, not across deployments, but the doubling of the collective share when the degree doubles is robust to that caveat.

The most direct finding is an absence. The engine’s startup warning at degree eight states that head dimension sharding of the four head layers may cause all to all communication overhead. Across both degree eight traces, spanning every compiled decode and prefill program the export captured, there is no all to all device operation: not one instance. Entries whose names contain the phrase do occur in raw traces, and all of them are compiler pass timings on the host, microseconds long and executed two or three times at compilation; the analysis pipeline classifies them as host work, and doing so is necessary to avoid manufacturing exactly the artifact the warning primes an analyst to find. Whatever the head dimension sharding path costs, on this stack, this model, and this mesh, it is not paid in all to all operations. The cost that does appear at the crossing is the per layer scaling degradation of Section 6.4, which requires no exotic collective to explain the warning’s substance: the warned layers are measurably the ones that stop scaling.

7 Discussion

What the crossing means for deployment. The practical statement of Equation 1 is that the tensor parallel degree of a deployment should be compared against the minimum key value head count of the model before any throughput projection is made. Below the minimum, the familiar accounting applies and the step’s dominant costs shrink together. At the crossing, the layers at the minimum stop scaling like the rest of the model, and the projection must carry two rates. For Gemma 4 31B at degree eight the head limited population is small, ten layers of sixty, and the aggregate consequence is the measured 1.81 times attention scaling against the ideal 2.0. For a model whose minimum head count is lower, or a deployment whose degree is higher, the head limited fraction

of the step grows, and the same measurement protocol applied there would quantify how fast.

What it means for model architecture. Key value head counts are chosen during model design for quality and for cache size. This study adds a third consideration that costs nothing at training time: the minimum head count of any layer sets the largest tensor parallel degree at which that layer's cache still shards fully. A hybrid model whose global layers carry the small head count, as here, concentrates the crossing cost in the layers that already read the most context. Designers who expect their models to be served at degree eight and beyond can read Table 4 as the per layer price list of that choice on current TPU serving software.

The premium is not a constant of the architecture. The fourth paper of this series measured the sliding window premium and bounded its mechanism at one parallel degree. This paper shows the premium is a function of the deployment: 58 percent at degree four, 4 percent at degree eight, negative under the stock block size. All three numbers are correct in their configurations. The general lesson for per layer cost measurements is that a layer type's cost is a property of the triple of layer, kernel configuration, and sharding, and that quoting it without the second and third coordinates invites transfer errors between deployments.

Warnings, banners, and evidence. Two results in this paper concern messages the stack prints. A warning promised all to all overhead that no trace contains; a banner promised an fp8 cache that no allocation delivered, in a format the allocation then contradicted when the feature was engaged explicitly. Neither message is wrong in an uninteresting way: the warning names a real mechanism whose measurable cost simply takes a different form (degraded per layer scaling rather than a new collective), and the banner names a real feature whose activation path differs from the advertised one. The operational rule the series keeps rediscovering is that printouts at the announcement layer are hypotheses, and the allocation lines, constructor echoes, and compiled kernel names at the acting layer are the observations.

Where the next throughput lives. The batch frontier ends at a factor of 1.09 per doubling, so the deployment's remaining aggregate headroom is inside the decode step. The measured step at batch 256 assigns roughly 5.6 ms to decode attention on the tuned kernel; per chip weight streaming at degree eight bounds a further fixed term; and the collective share doubles when the degree does. Of these, the term with a demonstrated intervention is attention, where the head limited layers now run within a few percent of the windowed ones and further gains require kernel or sharding work rather than configuration. The combined fp8 and batch result of Section 6.2 indicates the cache byte lever functions on this stack once engaged explicitly; its isolated contribution is the first item of the planned decomposition session.

8 Limitations

The claims are bounded by the following, and the bounds are part of the result.

The workload is an offline batch of short prompts with fixed 128 token outputs under greedy decoding, the decode dominated regime of the whole series. Services with long contexts, streaming arrivals, or sampling will land elsewhere; the fourth paper measured how the block size economics move with context length, and the same caution applies to every figure here.

The hardware scope is one single host v6e-8 slice. Multi host deployments cross to a different interconnect regime, and nothing here speaks to it. The degree comparison spans four and eight only; degree two does not fit this model, and degrees above eight require more hosts.

The batch 1,024 result changes two variables against the batch 512 row, cache precision and batch, plus the memory fraction required to run at all. It is reported as a combined operating point, and no decomposition of its gain is claimed.

The trace derived figures rest on profiled runs whose collective durations include synchronization waiting that profiling inflates, and on exports that captured roughly seven decode steps at degree eight and fourteen at degree four. The per layer attention means are tight (dozens of samples per operation, spreads near one percent) and their ratios drive the paper's central claims; per step communication totals are not quoted for exactly this reason, and the communication section is deliberately structural. The step counts also mean the traces sample the steady state early in the decode; costs that drift with sequence length over the 128 generated tokens are averaged accordingly.

The key value head census per layer is inferred from the architectural boundary and the measured scaling signature, because the configuration printout that would state it directly was itself version dependent. The inference is overdetermined by the data, but a stack whose reporting stabilizes would allow it to be stated from configuration alone.

9 Future Work

Three continuations are staged, in the sense that the harness for each already exists and is archived with this paper's code.

First, a decomposition session at batch 1,024: the same operating point with and without the fp8 cache, with and without the block override, and repeats of the winner, to attribute the combined gain of Section 6.2 to its variables and to add block 256 and batch 2,048 contrasts.

Second, the communication budget as a measurement rather than a structure: unprofiled step timing combined with per operation trace means from longer captures, to state per step collective time with the synchronization caveat quantified instead of avoided.

Third, the crossing at higher degrees. The prediction of Section 3 sharpens as the degree grows past the head count; a multi host campaign at degrees sixteen and thirty two, on

models chosen to place the crossing at different depths, would turn the two point measurement here into a curve, and would test whether the head limited layers’ 1.26 times gain at the first crossing decays further, as the mechanism implies.

10 Conclusion

On a single host TPU v6e-8, serving Gemma 4 31B with vLLM under a fixed one variable protocol, this study measured the first tensor parallel degree that crosses the model’s minimum key value head count. The kernel block size override of the earlier papers in this series transferred to multi chip serving with gains of 69 percent at degree four and 54 percent at degree eight; the batch frontier on the tuned kernel closed at 13,540 aggregate tokens per second at batch 1,024 with an fp8 cache verified at the allocation level. Per layer traces showed the crossing itself: doubling the degree sped up the fifty fully sharded windowed layers by 1.92 times per layer and the ten head limited global layers by 1.26 times, collapsing a 58 percent sliding window premium to 4 percent, while the all to all overhead the stack warned of never appeared as a device operation. Tensor parallelism sharded the model; it did not shard it uniformly, and the boundary of the non-uniformity is an architectural constant that model designers set and deployment planners can now price. Documented gaps between advertised and implemented behavior, and the truncation robust trace methodology that exposed several of them, accompany the numbers so that both can be checked.

Acknowledgments

Google ML Developer Programs and the Google Developers Program supported this work by providing Google Cloud credits.

Data and Code Availability

The measurement protocol, the run matrix, the benchmark and analysis scripts, the pinned environment specification, and the trace analysis tool with its truncation robust estimator are archived with this deposit (DOI: 10.5281/zenodo.21957969). Run logs and per run metrics are available from the author on reasonable request.

References

- [1] Joshua Ainslie et al. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [2] Noam Shazeer. Fast transformer decoding: One write-head is all you need. In *arXiv preprint arXiv:1911.02150*, 2019.
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [4] Albert Q. Jiang et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [5] Gemma Team. Gemma: Open models based on gemini research and technology. Google DeepMind technical report series, 2025.
- [6] Rubens A. Zimbres. From 1,540 to 19,511 tokens per second on a single tpu chip: A measurement study of llm inference optimization on tpu v5e, 2026.
- [7] Rubens A. Zimbres. The decode block size heuristic in tpu ragged paged attention reduces llm inference throughput by 28 to 69 percent, 2026.
- [8] Rubens A. Zimbres. Batch scaling and goodput of a tuned attention kernel on tpu v6e: Throughput, latency, energy, and cost measurements of vllm serving gemma 4 31b, 2026.
- [9] Rubens A. Zimbres. The sliding window premium: Throughput, goodput, and per layer attention costs of hybrid attention models at realistic context lengths on tpu v6e, 2026.
- [10] Mohammad Shoeybi et al. Megatron-lm: Training multi-billion parameter language models using model parallelism. In *arXiv preprint arXiv:1909.08053*, 2019.
- [11] Reiner Pope et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
- [12] Jacob Austin et al. How to scale your model: A systems view of llms on tpus. Online book, Google DeepMind, 2025.
- [13] Norman P. Jouppi et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
- [14] Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [15] Gyeong-In Yu et al. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [16] Amey Agrawal et al. Taming throughput-latency trade-off in llm inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

- [17] Yinmin Zhong et al. Distserve: Disaggregating pre-fill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [18] Tri Dao et al. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.